

Лабораторна робота №1

Застосування JavaScript для розв'язання прикладних задач за допомогою консолі браузера

Мета роботи: Навчитись застосовувати JavaScript для розв'язання прикладних задач за допомогою консолі браузера.

1.1 Теоретичні відомості

JavaScript (далі JS) – це мова програмування, що використовується в складі HTML-сторінок для збільшення їх функціональності та можливостей взаємодії з користувачем. JS є однією зі складових динамічного HTML. JS є мовою із динамічною типізацією (тип змінної визначається у процесі роботи (runtime) сценарію). Рушій JS усередині браузера працює послідовно (у один потік). JS-код виконується послідовно, згідно порядку включення його компонентів у текст html-файлу.

Якщо сторінка містить JS-сценарій, то з її JS-складовою можна взаємодіяти за допомогою консолі браузера. Зі сторони JS вивід у консоль забезпечується доступом до властивості `log` об'єкту консолі (`console.log()`), аргументом якої є повідомлення, змінна або об'єкт для виводу в консоль. З боку консолі можливо здійснювати виклик визначених у JS-сценарії сторінки функцій із довільними аргументами та спостерігати за їх виводом і поведінкою загалом. Для цього необхідно увести JS-код, який виконуватиме необхідні дії.

Усі математичні функції та константи, наприклад число π або тригонометричні функції є властивостями та методами вбудованого об'єкта `Math`. Наприклад, для застосування числа π достатньо звернутись до `Math.PI`.

1.2 Хід роботи

- 1.2.1 У створеному на GitHub репозиторії для курсу створити нову підпапку для завдання лабораторної.
- 1.2.2 Написати JavaScript-функцію `triangle`, яка розв'язує прямокутний трикутник (знаходить всі його сторони та обидва гострі кути) за двома

заданими елементами і їх «типами» (катет, прилеглий кут, гіпотенуза і т.д.) і яка може бути запущена у консолі для певних значень аргументів. На початку JS-сценарій повинен вивести за допомогою `console.log()` інструкцію з використання.

Перелік можливих «типів» значень наведено у таблиці 1:

Таблиця 1 – Позначення «типів» наявних елементів та пояснення до них

Позначення	Що позначає
leg	катет
hypotenuse	гіпотенуза
adjacent angle	прилеглий до катета кут
opposite angle	протилежний до катета кут
angle	один з двох гострих кутів (коли задана гіпотенуза)

Відповідно, функція повинна приймати аргументи у такому порядку:

значення аргумента 1, «тип» аргумента 1 (див. табл. 1), значення аргумента 2, «тип» аргумента 2 (див. табл. 1).

При цьому, передбачити всеможливі порядки слідування «типів» аргументів, наприклад:

```
triangle(4, "leg", 8, "hypotenuse");  
triangle(8, "hypotenuse", 4, "leg");
```

При вказанні величин кутів використовувати **градуси** (при цьому врахувати, що тригонометричні функції-методи `Math` оперують **радіанами**).

Вивід результатів здійснити за допомогою `console.log()`. При виведенні скористатись класичними математичними позначеннями сторін (c – гіпотенуза, a та b – катети, α – гострий кут, який лежить навпроти катета a , β – гострий кут, який лежить навпроти катета b).

Якщо функція успішно виконала обчислення, повернути „success”, якщо користувач увів щось неправильно (за винятком випадків некоректних значень, про що сказано у наступному абзаці), наприклад, увів «hypotenus» замість «hypotenuse» або увів несумісну пару «типів», то вивести повідомлення (console.log()) про те, що користувачу слід ще раз перечитати інструкцію та повернути „failed”.

Забезпечити перевірку на некоректність уведених значень (від’ємні або нульові аргументи, катет менший за гіпотенузу, негострі кути, тощо) при якій у випадку некоректних даних повертається рядок із відповідним повідомленням (за бажанням, можна додати вивід повідомлення у консоль).

Опціонально, можна визначити значення для аргументів та їх «типів» за замовчуванням.

Приклади тестових запусків наведено на рисунку 1:

```
>> triangle(7, "leg", 18, "hypotenuse");  
a = 7  
b = 16.583123951777  
c = 18  
alpha = 22.88538047615857  
beta = 67.11461952384143  
← "success"  
>> triangle(60, "opposite angle", 5, "leg");  
a = 5  
b = 2.8867513459481295  
c = 5.773502691896258  
alpha = 60  
beta = 30  
← "success"  
>> triangle(43.13, "angle", -2, "hypotenuse");  
← "Zero or negative input"
```

Рисунок 1 – Приклади тестових запусків

1.2.3 Надати викладачеві посилання на зроблену веб-сторінку, розміщену на сервері GitHub Pages (посилання повинне бути у форматі: <https://username.github.io/userrepo/userdirectory>), а також, посилання на безпосередньо сам репозиторій (посилання повинне бути у форматі: <https://github.com/username/userrepo>).

1.2.4 Зробити висновки щодо виконаної роботи.

1.3 Декларація про використання генеративного штучного інтелекту

Зразок: Під час виконання роботи було використано інструмент [Назва інструменту, наприклад: ChatGPT 4o / GitHub Copilot] для [вказати мету, наприклад: відлагодження помилки підключення до БД / оптимізації стилю коду / тестування самостійно написаного застосунку]. Увесь поданий результат реалізовано та оформлено здобувачем власноруч [за потреби можна уточнити верифікацію за джерелами/документацією]. Здобувач бере на себе повну відповідальність за коректність результатів та зміст лабораторної роботи.

1.4 Зміст звіту

1. Титульна сторінка та тема роботи.
2. Мета роботи.
3. Короткі теоретичні відомості (обсягом до 1 сторінки).
4. Хід роботи (результати виконаної роботи).
5. Декларація про використання генеративного штучного інтелекту.
6. Висновки.

1.5 Контрольні запитання

- 1.5.1 За що відповідає JavaScript у трійці HTML, CSS, JavaScript?
- 1.5.2 Що зумовлено тим, що JavaScript є мовою із динамічною типізацією?
- 1.5.3 Як задати змінну у JavaScript?
- 1.5.4 Як задати функцію у JavaScript?
- 1.5.5 Які існують області видимості у JavaScript?
- 1.5.6 Що таке scope chain і які його особливості?

1.5.7 Які типи даних визначено у JavaScript?

1.5.8 Поясніть різницю між операторами «==» та «===».